

DESPLIEGUE DE UN MODELO DE DEEP LEARNING PARA LA PREDICCIÓN DE
BAJO GASTO CARDIACO A PARTIR DE SEÑALES HEMODINÁMICAS EN UNA
UNIDAD DE CUIDADOS INTENSIVOS (DMDL-PC-SH-UCI)

DAVID ARTURO PEREZ DIAZ

Manual de Usuario

UNIVERSIDAD INDUSTRIAL DE SANTANDER
FACULTAD DE INGENIERÍAS FISICOMECAÑICAS
ESCUELA DE INGENIERÍAS
ELÉCTRICA, ELECTRÓNICA Y DE TELECOMUNICACIONES
BUCARAMANGA

2025

CONTENIDO

	pág.
LISTA DE FIGURAS	4
LISTA DE TABLAS	5
1 TERMINOLOGIA Y PROPUESTA METODOLOGICA.	6
1.1 Terminología	6
1.1.1 Conceptos Previos	6
1.1.2 Tecnologías	6
1.2 Propuesta metodológica	7
1.2.1 Objetivos	7
1.2.2 Metodología	7
2 INFRAESTRUCTURA DE LA SOLUCION.	9
2.1 Consideraciones generales	10
2.1.1 Formato de salida	12
2.1.2 Frameworks	12
2.1.3 Dependencias	12
2.2 Arquitectura interna	13
2.3 Artefactos	14
2.3.1 main.py	14
2.3.2 logger.py	17
2.3.3 predict.py	20
2.3.4 auth.py	23
2.3.5 database.py	26
2.3.6 models.py	28
2.3.7 middleware.py	30

3	Despliegue de Operaciones.	32
3.1	Ventajas de usar Docker	32
3.2	Contras de usar Docker	32
3.3	El archivo Dockerfile	33
3.4	El archivo <code>compose-config-swarm.yaml</code>	36
4	Endpoints y servicios.	38
4.1	Default	38
4.2	Inferencia	38
4.3	Autenticación	38
4.4	Parámetros de Entrada - Inferencia	39
4.5	Respuesta	40
4.6	Creación de usuario	41
4.7	Generador de sesión	43
5	Casos de usuario	45
5.0.1	Orquestador Docker Swarm	48
5.0.2	Jerarquía de directorios	49
5.0.3	Integración con el ambiente	50
5.1	Cliente	52
5.1.1	El archivo <code>client.py</code>	52
5.2	Pruebas de concurrencia y escalabilidad	56
5.2.1	El archivo <code>deployment_test.py</code>	57
5.2.2	Pruebas de integración	58
5.2.3	Pruebas de integración con Docker Swarm	62

LISTA DE FIGURAS

	pág.
Figura 1	Infra-estructura de la solucion 9
Figura 2	Interactividad interna 13
Figura 3	Distribucion interna 49
Figura 4	Archivo main_docker.sh 50
Figura 5	Archivo predict_docker.sh 51
Figura 6	Archivo main_cont.sh 51
Figura 7	Archivo predict_cont.sh 52
Figura 8	Logs de ejecucion - Canal del servidor 56
Figura 9	Logs de ejecucion - Canal del cliente 56
Figura 10	Reporte de recursos de hardware en tiempo real 59
Figura 11	Logs de ejecucion - Canal del cliente 59
Figura 12	Recursos de ejecucion - Canal del Servidor 60
Figura 13	Logs de ejecucion - Canal del cliente 60
Figura 14	Recursos de ejecucion - Canal del Servidor 61
Figura 15	Logs de ejecucion - Canal del Cliente 61
Figura 16	Recursos de ejecucion - Canal del Servidor 62
Figura 17	Logs de consola - Canal del Servidor 63
Figura 18	Logs de ejecucion - Canal del Cliente 63
Figura 19	Recursos de ejecucion - Canal del Servidor 64

LISTA DE TABLAS

pág.

1. TERMINOLOGIA Y PROPUESTA METODOLOGICA.

1.1. Terminología

1.1.1. Conceptos Previos Este documento asume el dominio de conceptos tales como:

1. Entornos de desarrollo.
2. Protocolo de comunicaciones HTTP.
3. Lógica asíncrona.
4. REST Application Programming Interface (API).
5. Protocolo IP.
6. Redes y puertos.
7. Encapsulado de ambientes.
8. Orquestación de servicios.

1.1.2. Tecnologías Durante los procesos de elaboración, implementación y despliegue se involucraron tecnologías como:

1. Python 3.11.0 .
2. SQLite.
3. JSON.
4. JWT.
5. HTTP.
6. OAuth 2.0.

7. Docker Swarm.
8. CLI Commands.

1.2. Propuesta metodológica

El presente trabajo de grado tiene como objetivo principal abordar el despliegue de un modelo de inteligencia artificial en una máquina virtual Ubuntu utilizando contenedores Docker, mediante la implementación de FastAPI. Este enfoque proporciona un entorno flexible y escalable para la ejecución de modelos de Deep Learning, facilitando su integración en aplicaciones y sistemas existentes.

1.2.1. Objetivos

- Desarrollar un proceso de despliegue eficiente y replicable para modelos de inteligencia artificial en entornos basados en Ubuntu.
- Implementar FastAPI como interfaz de programación para la comunicación entre el modelo de IA y las aplicaciones cliente.
- Utilizar contenedores Docker para garantizar la portabilidad y el aislamiento del entorno de ejecución del modelo.

1.2.2. Metodología Este documento busca exponer los objetivos planteados bajo las siguientes técnicas metodológicas:

1. Preparación del Entorno: Instalación y configuración de una máquina virtual Ubuntu con los requisitos necesarios para ejecutar contenedores Docker.
2. Desarrollo de una API utilizando FastAPI para exponer el modelo de Deep Learning y permitir su interacción con otras aplicaciones.
3. Creación de Contenedores Docker: Configuración de contenedores Docker para encapsular el modelo de Deep Learning y sus dependencias.

4. Despliegue y Pruebas: Implementación del modelo de Deep Learning en la máquina virtual Ubuntu y realización de pruebas para verificar su funcionamiento correcto.

2. INFRAESTRUCTURA DE LA SOLUCION.

El entorno dedicado a producción y operaciones deberá poder recibir consultas y estar en la capacidad de comunicar sus resultados. Para ello, se implementa el protocolo HTTP, de modo que existirá la relación cliente-servidor, lo cual hace necesario la declaración de puertos y reglas del host.

Usando contenedores, la virtualización del ambiente es una de las condiciones de operación, lo que conlleva la necesidad de asignar las relaciones pertinentes entre los puertos del ambiente virtualizado y la máquina local host, permitiendo así la vía libre de peticiones y respuestas entre los clientes de la aplicación (otras aplicaciones con acceso a la red privada) y la máquina local host (máquina que contiene el contenedor en ejecución).

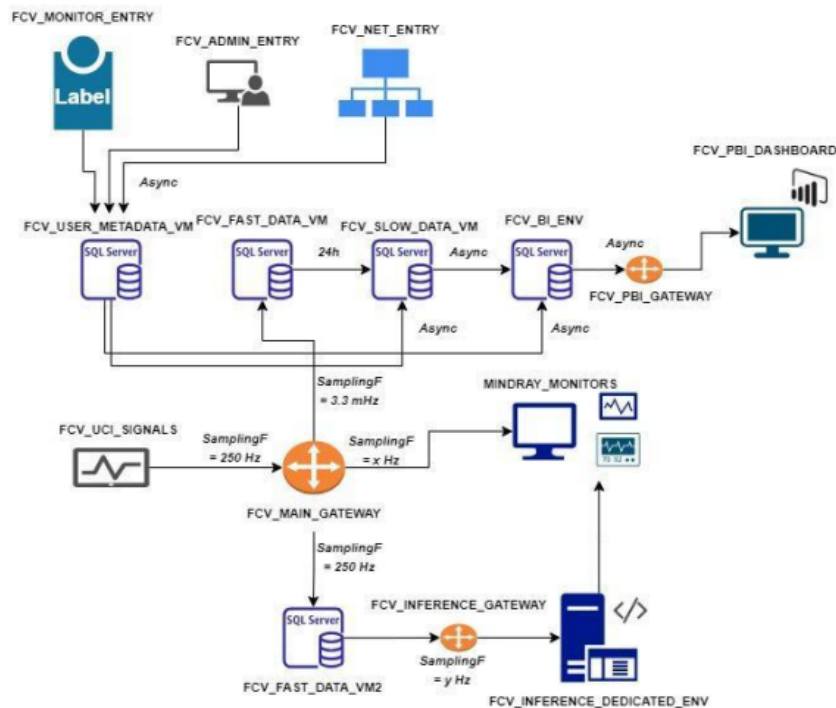


Figura 1. Infra-estructura de la solución

2.1. Consideraciones generales

El modelo de Deep Learning expuesto funcionará tomando como parámetros de entrada la información histórica hemodinámica de cada paciente por un rango de 1 hora con 48 minutos, esto quiere decir que para realizar una sesión de inferencia dado un paciente se necesitará la información hemodinámica del mismo por los últimos 108 minutos. El grupo de investigación en conectividad y procesamiento de señales CPS exportó los siguientes artefactos para establecer el acuerdo de servicio por el cual se define el formato de entrada al modelo de Deep Learning, proveer la versión oficial del modelo de Deep Learning que será desplegada y especificar los requerimientos tecnológicos y de ambiente que se requieren para la ejecución del modelo en materia.

Los artefactos en cuestión:

- LSTM.h5: El modelo de Deep Learning en formato Keras.
- predict.py: Artefacto para cargar y pre procesar datos, cargar el modelo y ejecutarlo.
- requirements.txt: Archivo de texto plano especificando dependencias del ambiente.

Tomando en cuenta que la frecuencia de muestreo de las señales hemodinámicas en cada paciente es de 1 cada 5 minutos, se prevé que entre cada sesión de inferencia se dé este mismo periodo de tiempo. Asumiendo un flujo de pacientes alto, es lícito concluir que la solución desplegada deberá responder eficientemente a consultas múltiples en el mismo o casi mismo instante de tiempo, es decir, deberá constituirse como una aplicación capaz de recibir, procesar y responder consultas múltiples por uno o múltiples usuarios.

Cualesquiera consultas realizadas por determinado cliente deberán incluir consigo la información requerida en concepto de parámetros de entrada para la sesión de inferencia. El formato de estos datos se constituye de la siguiente manera:

Listado 2.1. Objeto de entrada - parametros de inferencia

```
1 data_input = {'idAtencion':int,  
2             'idSigno':int,  
3             'nomSigno':str,  
4             'valor':float,  
5             'fecRegistro':str  
6             }
```

El artefacto predict.py sugiere que la forma final en la que deberán ser ingestados los datos proviene de las columnas nomSigno, que es la que contiene el nombre de las variables hemodinámicas, y la columna valor, que es la que contiene la información numérica dada la variable hemodinámica. El formato de los datos necesario es tal que estas variables ahora sean columnas independientes cada una con su respectivo nombre y valor cuando sea el caso y cuando no, consignar un NaN para posterior filtrado, tal que solo quedarán registros efectivos (no NaN) distribuidas las variables hemodinámicas en columnas.

El formato de entrada que comprende el modelo de inferencia contempla un arreglo de datos que a través de sus dimensiones segmenta los registros por ID de usuario, variables hemodinámicas y cantidad de registros requeridos por el batch de inferencia (30 registros) por cada variable.

La salida del modelo de inferencia asocia una ponderación de probabilidad de 0 a 1, 0 siendo la menor de las ponderaciones (caso menos probable) y 1 constituyéndose como la probabilidad más predominante. Cada sesión de inferencia ponderará cuatro diferentes casos probables relacionados con el estado rítmico del paciente, dichos estados son

Arresto Cardíaco, Bajo Gasto Cardíaco, Sano y Shock Cardiogénico.

2.1.1. Formato de salida El servicio expuesto para la sesión de inferencia tendrá un formato de salida tal que se pueda identificar el paciente asociado a dicha sesión de inferencia, las ponderaciones producidas por la sesión de inferencia y el estado con la mayor probabilidad asociada. Esta información será expuesta como una respuesta a una consulta HTTP mediante un objeto json.

Sírvase el siguiente como ejemplar del formato de salida:

Listado 2.2. Objeto de salida - respuesta del sistema

```
1 response_object = {"idAtencion": 0,  
2                   "inferences": ["string"],  
3                   "State": "string"}
```

El formato de salida propone la segmentación de resultados a nivel de usuario por lo que se le podría solicitar al servicio de inferencia realizar la inferencia bien masivamente a un número múltiple de usuarios o una inferencia singular enfocada a un solo usuario. La respuesta del servicio de inferencia sería emitida en un documento json al cliente, el cual podría disponer del mismo como lo requiera.

2.1.2. Frameworks La implementación del servidor se constituye mediante el framework FastAPI con el que se suplen requerimientos como tasas de consulta multitudinales, compatibilidad JSON, seguridad mediante el uso de tokens JWT para autenticar al cliente, así como permitir que el cliente pueda autenticar las respuestas del servidor. El token se empleará para validar al cliente durante el acceso al servicio, y cada cliente, a partir de sus credenciales, dispondrá de un token único para emitir consultas al servidor.

2.1.3. Dependencias Las dependencias y configuración del ambiente se prevén cubiertas al contemplar el despliegue de la aplicación como un contenedor Docker. Sin embargo, es importante aclarar que para el correcto funcionamiento de Docker se hace necesario contar con óptimas cantidades de memoria RAM, ya que este funciona basado en una máquina virtual.

También es mandatorio garantizar el acceso a este recurso mediante conexiones locales

o de red local. En otras palabras, se debe resolver la relación cliente-servidor en torno a identidades y métodos de autenticación.

Como se ha mencionado previamente, otro de los requerimientos para el correcto funcionamiento es proveer al host con el motor Docker, por lo que se hace necesaria la instalación del mismo antes de suponer cualquier despliegue.

2.2. Arquitectura interna

La infraestructura interna de la aplicación y como esta interactúa con sus diferentes componentes queda descrita por el siguiente diagrama de conexiones:

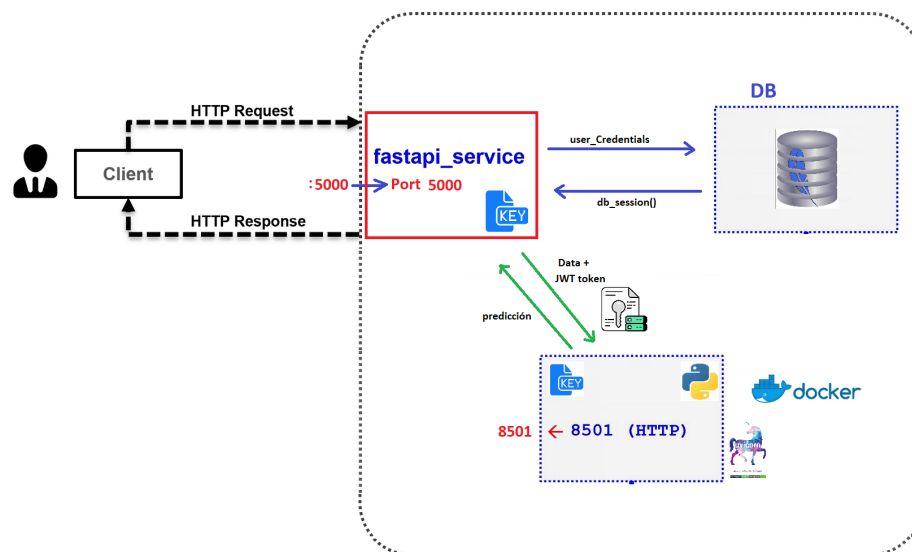


Figura 2. Interactividad interna

Bajo el protocolo de funcionamiento de una REST API, el cliente se comunica enviando peticiones HTTP a los dos diferentes componentes de la aplicación: gestión de sesión e inferencia. Cada aplicación trabaja con una imagen diferente, por lo que son dos contenedores distintos interactuando directamente con las solicitudes del cliente. Docker Swarm activa sus funcionalidades sobre el aplicativo de inferencia, creando réplicas del mismo y ejecutándolas en una red interna, permitiendo la paralelización de las solicitudes y aprovechando las capacidades de hardware del servidor.

2.3. Artefactos

2.3.1. main.py La aplicación está construida utilizando FastAPI y proporciona servicios de autenticación, creación de usuario y comprobación de estado del modelo.

Inicialización de la Aplicación:

Listado 2.3. App Init

```
1 app1 = FastAPI()
2 app1.include_router(auth.router)
3 models.Base.metadata.create_all(bind=engine)
```

Se crea una instancia de FastAPI y se incluye el router de autenticación. Además, se inicializa la base de datos creando todas las tablas definidas en `models`.

Dependencias de la Base de Datos:

Listado 2.4. Creador de Sesion

```
1 def get_db():
2     db = SessionLocal()
3     try:
4         yield db
5     finally:
6         db.close()
7
8 db_dependency = Annotated[Session, Depends(get_db)]
9 user_dependency = Annotated[dict, Depends(get_current_user)]
```

Se define una función para obtener la sesión de la base de datos y cerrar la conexión después de su uso. También se crean dependencias anotadas para la base de datos y el usuario autenticado.

Middleware:

Listado 2.5. Accionador Middleware

```
1 appl.add_middleware(BaseHTTPMiddleware, dispatch=middleware_log)
```

Se agrega un middleware personalizado para el logging de peticiones HTTP.

Definición de Modelos:

```
class BatchIn(BaseModel):
```

```
    idAtencion: int
```

```
    idSigno: dict
```

```
    nomSigno: dict
```

```
    valor: dict
```

```
    fecRegistro: dict
```

```
class PredictionOut(BaseModel):
```

```
    idAtencion: int
```

```
    inferences: dict
```

```
    State: str
```

Se definen los modelos `BatchIn` y `PredictionOut` utilizando `pydantic`. `BatchIn` representa la entrada para las predicciones, mientras que `PredictionOut` representa la salida.

Ruta Principal:

Listado 2.6. Default Endpoint

```
1 @app.get("/", status_code=status.HTTP_200_OK)
2 async def user(payload: dict):
3     """
4     Retrieve user information.
5
6     Parameters:
7     - user: The user object containing user information.
8     - db: The database dependency.
9
10    Returns:
11    - A dictionary containing the user information.
12
13    Raises:
14    - HTTPException with status code 401 if authentication fails.
15    """
16    if isValidToken(payload.get('token')) is False:
17        raise HTTPException(status_code=401, detail="Token is not valid.")
18    else:
19        payload = jwt.decode(payload.get('token'), SECRET_KEY, algorithms=[ALGORITHM])
20        username: str = payload.get('sub')
21    return {"User": username}
```

Esta ruta verifica la autenticación del usuario y retorna la información del mismo si está autenticado.

Ruta de Comprobación de Estado:

Listado 2.7. Health Check

```
1 @app.get("/home")
2 async def home():
3     """
4     Endpoint for the home page.
5
```



```

6 Returns:
7     dict: A dict containing the health check status and model version.
8     """
9     await asyncio.sleep(0) # type: ignore
10    model_version = "1.0.0"
11    return {"health_check": "OK", "model_version1": model_version}

```

Esta ruta proporciona una comprobación de estado del modelo, retornando un mensaje de salud y la versión del modelo.

Ruta de Predicción:

Listado 2.8. Ruta de inferencia

```

1 @app2.post("/models", response_model=PredictionOut)

```

Esta ruta recibe una entrada de tipo `BatchIn`, verifica la autenticación del usuario, y genera inferencias basadas en los datos proporcionados. Las inferencias se retornan en el formato definido por `PredictionOut`.

2.3.2. `logger.py` Este artefacto contiene las pautas y configuraciones para el registro de la ejecución y su trazabilidad.

El siguiente código establece un sistema de logging en Python que permite registrar mensajes de log tanto en la consola (`stdout`) como en un archivo llamado `app.log`. A continuación, se describe la funcionalidad y el razonamiento detrás de cada parte del código.

Listado 2.9. Importacion y creacion del logger

```
1 import logging
2 import sys
3
4 logger = logging.getLogger(__name__)
5 formatter = logging.Formatter(fmt="%(asctime)s , %(levelname)s , %(message)s")
6 stream_handler = logging.StreamHandler(sys.stdout)
7 file_handler = logging.FileHandler('app.log')
8 stream_handler.setFormatter(formatter)
9 file_handler.setFormatter(formatter)
10 logger.handlers = [stream_handler, file_handler]
11 logger.setLevel(logging.INFO)
```

Importación de Módulos:

- `import logging`: Importa el módulo de logging de Python, que proporciona una interfaz flexible para emitir mensajes de log desde programas Python.
- `import sys`: Importa el módulo sys, que proporciona acceso a algunas variables utilizadas o mantenidas por el intérprete de Python y a funciones que interactúan fuertemente con el intérprete.

Creación del Logger:

```
logger = logging.getLogger(__name__)
```

Esta línea crea un objeto logger. El nombre del logger se establece en `__name__`, lo que permite que el nombre del logger sea el nombre del módulo en el cual se encuentra este código. Esto es útil para identificar el origen de los mensajes de log en aplicaciones más grandes.

Definición del Formato del Log:

```
formatter = logging.Formatter(fmt="%(asctime)s , %(levelname)s , %(message)s")
```

Se crea un objeto formatter que define el formato de los mensajes de log. El formato especificado incluye:

- `%(asctime)s`: La fecha y hora en que se generó el mensaje de log.
- `%(levelname)s`: El nivel de severidad del mensaje de log (por ejemplo, INFO, WARNING, ERROR).
- `%(message)s`: El contenido del mensaje de log.

Configuración de los Handlers:

```
stream_handler = logging.StreamHandler(sys.stdout)
file_handler = logging.FileHandler('app.log')
```

Se crean dos handlers:

- `stream_handler` Envía los mensajes de log a la consola (stdout).
- `file_handler` Envía los mensajes de log a un archivo llamado `app.log`.

Asignación del Formato a los Handlers:

```
stream_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)
```

Se asigna el objeto formatter creado previamente a ambos handlers, asegurando que los mensajes de log tengan el mismo formato sin importar su destino.

Asignación de los Handlers al Logger:

```
logger.handlers = [stream_handler, file_handler]
```

Se asignan los handlers configurados al logger. Esto permite que cualquier mensaje de log emitido por el logger se envíe tanto a la consola como al archivo de log.

Configuración del Nivel de Log:

```
logger.setLevel(logging.INFO)
```

Se establece el nivel de log del logger en `INFO`. Esto significa que el logger emitirá mensajes de log con un nivel de severidad de `INFO` o superior (por ejemplo, `WARNING`, `ERROR`, `CRITICAL`).

2.3.3. `predict.py` Esta sección describe las funcionalidades y el razonamiento detrás del código proporcionado, el cual realiza predicciones de estados cardíacos basadas en datos de pacientes utilizando un modelo de red neuronal LSTM.

Este artefacto se constituye inicialmente como la aplicación 2 (`app2`) exponiendo dos endpoints en su operativa, uno de ellos `http://172.30.19.97:5001/` el cual tan solo hace log del inicio de la aplicación y de la versión del modelo de Deep Learning cargado para producción. El segundo endpoint `http://172.30.19.97:5001/models` está definido en este artefacto y es el encargado exclusivo de realizar la inferencia.

Dependencias:

El código utiliza varias bibliotecas y dependencias que se deben importar al inicio:

- `pandas`: Para la manipulación de datos.
- `numpy`: Para operaciones numéricas.
- `StandardScaler` de `sklearn.preprocessing`: Para la normalización de los datos.
- `load_model` de `tensorflow.keras.models`: Para cargar el modelo LSTM pre entrenado.
- `asyncio`: Para manejar operaciones asíncronas.
- `Path` de `pathlib`: Para manejar rutas de archivos.

Inicialización:

El código inicializa el modelo LSTM y define un diccionario de clases:

Listado 2.10. Categorías de salida

```
1 model = load_model('LSTM--1.0.0.h5')
2 diccionario_clases = {
3     0: 'Arresto_cardiaco',
4     1: 'Bajo_gasto_cardiaco',
5     2: 'Sano',
6     3: 'Shock_cardiogenico'
7 }
```

Función predict:

La función `predict` es una función asíncrona que toma los datos de entrada y realiza predicciones. A continuación, se describen los pasos principales de esta función:

Creación del DataFrame

Se crea un DataFrame a partir de los datos de entrada:

```
df = pd.DataFrame(data)
```

Reorganización del DataFrame

El DataFrame se reorganiza utilizando la función `pivot_table` para estructurarlo según las especificaciones dadas:

Listado 2.11. Extracción de datos

```
1 df_resultante = df.pivot_table(index=['idAtencion', 'fecRegistro'],
2                                 columns='nomSigno', values='valor').reset_index()
```

Filtrado de Datos

Se filtran los datos para incluir solo las columnas deseadas y se utilizan los últimos datos registrados por cada paciente:

Listado 2.12. Características de inferencia.

```
1 columnas_deseadas = ['idAtencion', 'fecRegistro', 'MDC_BLD_PERF_INDEX',
2                       'MDC_ECG_HEART_RATE', 'MDC_ECG_V_P_C_RATE',
3                       'MDC_LEN_BODY_ACTUAL', 'MDC_MASS_BODY_ACTUAL',
4                       'MDC_PULS_OXIM_PULS_RATE', 'MDC_PULS_OXIM_SAT_O2',
5                       'MDC_TEMP', 'MDC_TTHOR_RESP_RATE']
6 df_resultante_filtrado = df_resultante[columnas_deseadas]
7 ultimas_20_filas_por_id = df_resultante_filtrado.groupby('idAtencion').tail(20)
8 df_resultante_lleno = ultimas_20_filas_por_id.ffill()
```

Selección de Características

Se seleccionan las características relevantes para la predicción:

Listado 2.13. Parametros de entrada

```
1 selected_features = ['MDC_BLD_PERF_INDEX', 'MDC_ECG_HEART_RATE', 'MDC_ECG_V_P_C_RATE',
2                       'MDC_LEN_BODY_ACTUAL', 'MDC_MASS_BODY_ACTUAL',
3                       'MDC_PULS_OXIM_PULS_RATE',
4                       'MDC_PULS_OXIM_SAT_O2', 'MDC_TEMP', 'MDC_TTHOR_RESP_RATE']
5 features = df_resultante_lleno[selected_features]
```

Creación de Secuencias

Se convierten los datos a secuencias por paciente:

Listado 2.14. Transformacion de los datos

```
1 sequences = []
2 patient_ids = df_resultante_lleno['idAtencion'].unique()
3 for patient_id in patient_ids:
4     patient_data = features[df_resultante_lleno['idAtencion'] == patient_id]
5     sequences.append(patient_data.values)
```

Normalización de los Datos

Se normalizan los datos utilizando StandardScaler:

Listado 2.15. Escalamiento de los datos

```
1 scaler = StandardScaler()
2 X_test = [scaler.fit_transform(seq) for seq in sequences]
```

```
3 X_test = np.reshape(X_test, (len(X_test), X_test[0].shape[0], X_test[0].shape[1]))
```

Predicción

Se realizan las predicciones utilizando el modelo LSTM cargado:

Listado 2.16. Ejecucion y produccion de predicciones

```
1 results = pd.DataFrame()
2 for i in range(len(X_test)):
3     resultados = model.predict(X_test[i:])
4     max_index = np.argmax(resultados)
5     state = diccionario_clases[max_index]
6     temp = pd.DataFrame({'idAtencion': patient_ids[i], 'Inference': [resultados],
7                          "State": state}, index=[0])
7     results = pd.concat([results, temp])
```

Se imprime la clase predicha para cada paciente:

Listado 2.17. Presentacion de resultados

```
1 for idx, resultado in enumerate(resultados, start=1):
2     clase_predicha = np.argmax(resultado)
3     nombre_clase_predicha = diccionario_clases[clase_predicha]
4     print(f"Prediccin {idx}: El paciente con id {patient_ids[i]} se encuentra en estado:
        {nombre_clase_predicha}")
```

Retorno de Resultados

Finalmente, la función retorna los resultados obtenidos:

```
return results
```

2.3.4. auth.py Esta sección describe las funcionalidades y el razonamiento detrás de un conjunto de códigos implementados con FastAPI, SQLAlchemy y otras librerías de autenticación y encriptación. El objetivo principal del código es proporcionar un sistema de autenticación basado en tokens para una aplicación web.

Importaciones y Configuraciones Iniciales:

El código comienza con las importaciones necesarias de varias librerías:

- `sqlalchemy`: Para trabajar con la base de datos.
- `datetime`: Para manejar fechas y tiempos.
- `typing`: Para anotaciones de tipos.
- `fastapi`, `pydantic`, `starlette`: Para la creación de la API y manejo de solicitudes y respuestas.
- `passlib.context`: Para la gestión de la encriptación de contraseñas.
- `jose.jwt`: Para la creación y verificación de JSON Web Tokens (JWT).

Además, se configura el router de FastAPI y se definen constantes para la clave secreta y el algoritmo de encriptación.

Definición de Clases:

- `CreateUserRequest`: Clase que define el esquema de solicitud para la creación de un usuario, con los campos `username` y `password`.
- `Token`: Clase que define el esquema de respuesta para el token de acceso, con los campos `access_token` y `token_type`.

Gestión de la Base de Datos:

La función `get_db` se encarga de proporcionar una sesión de base de datos para las solicitudes entrantes. Utiliza un contexto de gestión para asegurar que la conexión se cierre correctamente después de cada uso.

Creación de Usuarios:

El endpoint `create_user` permite crear un nuevo usuario. Toma un objeto `CreateUserRequest` y guarda el usuario en la base de datos con la contraseña encriptada.

Autenticación y Generación de Tokens:

El endpoint `login_for_access_token` se utiliza para autenticar a los usuarios y generar un token de acceso. Si las credenciales son válidas, se genera un JWT con una expiración de 20 minutos.

Funciones de Ayuda:

- `authenticate_user`: Verifica las credenciales del usuario contra la base de datos.
- `create_access_token`: Crea un JWT con el nombre de usuario y el ID del usuario, y una fecha de expiración.
- `get_current_user`: Decodifica el token JWT para obtener el usuario actual.

Razonamiento:

El código sigue buenas prácticas de desarrollo de software, incluyendo:

- Uso de dependencias para gestionar la inyección de dependencias (Depends).
- Separación de responsabilidades mediante funciones y clases bien definidas.
- Manejo seguro de contraseñas con encriptación.
- Generación y verificación de tokens seguros para la autenticación.

2.3.5. database.py Esta sección explica las funcionalidades y el razonamiento detrás del siguiente código en Python que utiliza SQLAlchemy para la gestión de una base de datos SQLite.

Listado 2.18. Base de datos

```
1 import sqlalchemy
2 from sqlalchemy import create_engine
3 from sqlalchemy.orm import sessionmaker
4 from sqlalchemy.ext.declarative import declarative_base
5
6 SQLALCHEMY_DATABASE_URL = 'sqlite:///./todosapp.db'
7
8 engine = create_engine(SQLALCHEMY_DATABASE_URL,
9                        connect_args={'check_same_thread': False})
10 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
11
12 Base = declarative_base()
```

Importación de Módulos:

- `import sqlalchemy`: Importa el paquete principal de SQLAlchemy, una biblioteca de SQL para Python.
- `from sqlalchemy import create_engine`: Importa la función `create_engine`, que se utiliza para configurar la conexión a la base de datos.
- `from sqlalchemy.orm import sessionmaker`: Importa `sessionmaker`, una función que se utiliza para crear nuevas sesiones de base de datos.
- `from sqlalchemy.ext.declarative import declarative_base`: Importa `declarative_base`, una fábrica que se utiliza para construir una clase base para modelos declarativos.

Configuración de la URL de la Base de Datos:

```
SQLALCHEMY_DATABASE_URL = 'sqlite:///./todosapp.db'
```

Se define una URL para la base de datos SQLite. Esta URL indica que se utilizará una base de datos SQLite local llamada `todosapp.db` que se ubicará en el mismo directorio que el script Python.

Creación del Motor de la Base de Datos:

Listado 2.19. Motor de base de datos

```
1 engine = create_engine(SQLALCHEMY_DATABASE_URL,  
2                        connect_args={'check_same_thread':False})
```

La función `create_engine` crea un motor de base de datos que se utilizará para interactuar con la base de datos SQLite. El argumento `connect_args` es específico para SQLite y permite que la base de datos sea accedida desde múltiples hilos.

Creación de la Sesión de la Base de Datos:

Listado 2.20. Generacion de la sesion local

```
1 SessionLocal = sessionmaker(autocommit=False,autoflush=False, bind=engine)
```

La función `sessionmaker` se utiliza para crear una clase de sesión. Esta clase se utilizará posteriormente para crear instancias de sesión de base de datos. Los argumentos `autocommit=False` y `autoflush=False` configuran la sesión para no realizar commits ni flush automáticos. El argumento `bind=engine` vincula la sesión al motor de la base de datos creado previamente.

Declaración de la Clase Base:

```
Base = declarative_base()
```

La función `declarative_base` se utiliza para crear una clase base a partir de la cual se definirán todas las clases de modelo. Esta clase base contiene metadatos que se utilizarán para crear las tablas en la base de datos.

2.3.6. models.py El siguiente código define un modelo de base de datos utilizando SQLAlchemy, una biblioteca ORM (Object-Relational Mapping) para Python. Este modelo representa la tabla `users` en la base de datos.

Users:

Listado 2.21. Clase Usuarios

```
1 from database import Base
2 from sqlalchemy import Column, Integer, String
3
4 class Users(Base):
5     __tablename__ = 'users'
6
7     id = Column(Integer, primary_key=True, index=True)
8     username = Column(String, unique=True)
9     hashed_password = Column(String)
```

Importaciones:

- `from database import Base`: Importa la clase `Base` desde el módulo `database`. Esta clase es la base para todos los modelos ORM en SQLAlchemy.
- `from sqlalchemy import Column, Integer, String`: Importa las clases necesarias para definir las columnas de la tabla y sus tipos de datos.

Definición de la Clase Users:

La clase `Users` hereda de `Base` y representa la tabla `users` en la base de datos.

Atributos de Clase:

- `__tablename__ = 'users'`: Define el nombre de la tabla en la base de datos que este modelo representa.
- `id = Column(Integer, primary_key=True, index=True)`: Define la columna `id` como un entero, clave primaria, e indexada para optimizar las búsquedas.

- `username = Column(String, unique=True)`: Define la columna `username` como una cadena de texto que debe ser única, asegurando que no haya dos usuarios con el mismo nombre de usuario.
- `hashed_password = Column(String)`: Define la columna `hashed_password` como una cadena de texto que almacena la contraseña del usuario de forma encriptada.

Razonamiento del Código:

- **Herencia de Base**: Heredar de `Base` permite que la clase `Users` se integre con SQLAlchemy, permitiendo la creación, consulta, actualización y eliminación de registros en la tabla `users`.
- **Uso de `__tablename__`**: Especificar `__tablename__` asegura que SQLAlchemy sepa a qué tabla en la base de datos se refiere este modelo.
- **Definición de Columnas**:
 - `id`: Utilizar una clave primaria es esencial para identificar de manera única cada registro en la tabla.
 - `username`: Hacer que esta columna sea única impide la duplicación de nombres de usuario, lo cual es crucial para la autenticación y gestión de usuarios.
 - `hashed_password`: Almacenar la contraseña de forma encriptada mejora la seguridad, protegiendo la información sensible de los usuarios.

2.3.7. middleware.py Esta sección describe la funcionalidad de un script de middleware en Python que utiliza FastAPI para registrar información sobre las solicitudes HTTP entrantes. El script hace uso de módulos como `asyncio`, `datetime` y `time`, así como un módulo personalizado de `logger` para llevar a cabo el registro de eventos. **Descripción del Script:** A continuación, se presenta el código del script:

Listado 2.22. Creación de una instancia de middleware

```
1 import asyncio
2 from fastapi import Request
3 from logger import logger
4 from datetime import datetime
5 import time
6
7 async def middleware_log(request: Request, call_next):
8     start = time.time()
9     response = await call_next(request)
10    duration = time.time() - start
11    log_dict = {
12        'url': request.url.path,
13        'method': request.method,
14        'date': datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
15        'duration': duration
16    }
17    logger.info(log_dict, extra=log_dict)
18    return response;
```

Funcionalidad: El script proporciona la funcionalidad de middleware en una aplicación FastAPI, con el objetivo de registrar detalles de cada solicitud HTTP que llega al servidor. La funcionalidad del script se describe a continuación:

- **Importación de Módulos:** Se importan los módulos necesarios:
 - `asyncio` para manejar funciones asíncronas.
 - `fastapi.Request` para manejar las solicitudes HTTP.
 - `logger` desde un módulo personalizado para el registro de eventos.

- `datetime` y `time` para manejar y registrar fechas y tiempos.
- **Función Asíncrona:** Se define una función asíncrona `middleware_log` que toma como parámetros una solicitud (`request`) y una función (`call_next`) que procesa la solicitud.
 - **Medición del Tiempo de Ejecución:** Al inicio de la función, se registra el tiempo de inicio (`start`) utilizando la función `time.time()`.
 - **Llamada a la Siguiente Función del Middleware:** Se llama a la función `call_next` para procesar la solicitud y se espera la respuesta (`response`).
 - **Duración de la Solicitud:** Se calcula la duración de la solicitud restando el tiempo de inicio del tiempo actual después de procesar la solicitud.
 - **Creación del Diccionario de Log:** Se crea un diccionario `log_dict` que contiene:
 - `url`: La ruta de la solicitud.
 - `method`: El método HTTP utilizado (GET, POST, etc.).
 - `date`: La fecha y hora actual en formato `dd/mm/AAAA HH:MM:SS`.
 - `duration`: La duración de la solicitud en segundos.
 - **Registro de la Información:** Se utiliza el `logger` para registrar la información del diccionario de log.
 - **Devolución de la Respuesta:** Finalmente, se devuelve la respuesta obtenida de la función `_next`.

3. Despliegue de Operaciones.

Docker es una plataforma de contenedorización que permite a los desarrolladores empaquetar una aplicación y sus dependencias en un contenedor, garantizando que la aplicación se ejecute de manera consistente en cualquier entorno. Los contenedores son ligeros y portátiles, proporcionando un aislamiento eficiente de las aplicaciones y sus entornos. Docker se utiliza ampliamente para el desarrollo, prueba y despliegue de aplicaciones, facilitando la integración continua y la entrega continua (CI/CD).

3.1. Ventajas de usar Docker

- **Portabilidad:** Docker permite empaquetar la aplicación junto con todas sus dependencias en un contenedor, asegurando que funcione de manera consistente en cualquier entorno.
- **Aislamiento:** Cada contenedor es independiente, lo que evita conflictos de dependencias y facilita el mantenimiento y la actualización de componentes individuales.
- **Escalabilidad:** Docker facilita la escalabilidad horizontal, permitiendo desplegar múltiples instancias de la aplicación para manejar aumentos en la carga utilizando el servicio de orquestación Docker Swarm.
- **Eficiencia:** Los contenedores son más ligeros que las máquinas virtuales, lo que reduce el uso de recursos y mejora el rendimiento.
- **Rapidez en el despliegue:** La creación y despliegue de contenedores es rápida, lo que acelera los ciclos de desarrollo y despliegue.

3.2. Contras de usar Docker

- **Complejidad Inicial:** Configurar y gestionar contenedores puede tener una curva de aprendizaje pronunciada para aquellos que no están familiarizados con Docker.

- **Persistencia de Datos:** Manejar el almacenamiento persistente puede ser complicado, ya que los datos dentro de los contenedores no son persistentes por defecto.
- **Seguridad:** Aunque Docker proporciona aislamiento, compartir el núcleo del sistema operativo puede presentar riesgos de seguridad si no se configuran adecuadamente las políticas de seguridad.
- **Rendimiento:** En algunos casos, el overhead de la capa de abstracción de Docker puede impactar el rendimiento en comparación con la ejecución directa en el host.

3.3. El archivo Dockerfile

El Dockerfile es un script que contiene una serie de instrucciones para crear una imagen de Docker. A continuación se muestra y explica cada instrucción del Dockerfile para la creación de imagen del aplicativo destinado a la inferencia.

```
1  # Usa una imagen base de Python
2  FROM python:3.11-slim
3
4  # Establece el directorio de trabajo en el contenedor
5  WORKDIR /app
6
7  # Copia el archivo de requisitos y luego instálalos
8  COPY requirements.txt .
9  RUN pip install --no-cache-dir -r requirements.txt
10
11 # Copia todo el contenido del directorio actual en el contenedor
12 COPY . .
13
14 # Exponer el puerto en el que correr la aplicación
15 EXPOSE 5000
16
17 # Comando para correr la aplicación
```

```
18 # CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "80"]
19 # CMD [ "python", "routes/predict.py" ]
20 CMD ["uvicorn", "app.routes.predict:app2", "--host", "0.0.0.0", "--port",
    "5000"]
```

Listado 3.1. Dockerfile

- **FROM:** Esta instrucción especifica la imagen base desde la cual se creará nuestra nueva imagen. En este caso, se utiliza `python:3.8-slim`, una versión ligera de Python 3.8.
- **WORKDIR:** Establece el directorio de trabajo dentro del contenedor en `/app`.
- **COPY:** Copia el archivo `requirements.txt` desde el sistema host al directorio de trabajo del contenedor.
- **RUN:** Ejecuta un comando dentro del contenedor. En este caso, instala las dependencias listadas en `requirements.txt` utilizando `pip`.
- **COPY:** Copia todos los archivos del directorio actual del host al directorio de trabajo del contenedor.
- **CMD:** Especifica el comando por defecto que se ejecutará cuando se inicie un contenedor a partir de esta imagen. Aquí, se ejecuta el script `app.py` utilizando Python.

El archivo `.dockerignore` especifica los archivos y directorios que Docker debe ignorar al copiar archivos a la imagen. Esto puede ayudar a reducir el tamaño de la imagen y a mejorar la seguridad.

```
1 __pycache__
2 *.pyc
3 *.pyo
4 *.pyd
5 .env
6 .git
7 .vscode
```

Listado 3.2. `.dockerignore`

- `__pycache__`: Carpeta donde se almacenan los archivos bytecode compilados de Python.
- `*.pyc`, `*.pyo`, `*.pyd`: Archivos bytecode de Python que no son necesarios para el contenedor.
- `.env`: Archivos de variables de entorno que pueden contener información sensible.
- `.git`: El directorio Git que contiene la historia del repositorio, generalmente no necesario dentro del contenedor.
- `.vscode`: Directorio de configuración del editor Visual Studio Code, no relevante para la ejecución en contenedor.

3.4. El archivo `compose-config-swarm.yml`

El archivo `compose-config-swarm.yml` se utiliza para definir y ejecutar aplicaciones Docker multi-contenedor, clave para despliegues de alta concurrencia y desempeño elevado. A continuación se presenta un ejemplo y su explicación.

```
1
2 services:
3   api:
4     image: api\_image:latest
5     ports:
6       - "5001:5001"
7     networks:
8       - app-network
9     deploy:
10      replicas: 3
11      update\_config:
12        parallelism: 2
13        delay: 10s
14      resources:
15        limits:
16          cpus: '1'
17          memory: 1024M
18      restart\_policy:
19        condition: on-failure
20
21 visualizer:
22   image: dockersamples/visualizer
23   ports:
24     - 8080:8080
25   volumes:
```

```

26 - /var/run/docker.sock:/var/run/docker.sock:ro
27 deploy:
28     placement:
29         constraints: [node.role == manager]
30
31
32 networks:
33     app-network:
34         driver: overlay

```

Listado 3.3. docker-version: '3.8'

- **version:** Especifica la versión del formato de `docker-compose.yml` que se está utilizando.
- **services:** Define los servicios que se ejecutarán. En este caso, solo hay un servicio llamado `app`.
- **app/build/context:** Indica el contexto de construcción, que es el directorio actual `(.)`.
- **app/build/dockerfile:** Especifica el archivo `Dockerfile` a utilizar para la construcción.
- **app/ports:** Mapea el puerto 8000 del contenedor al puerto 8000 del host.
- **app/volumes:** Monta el directorio actual del host en el directorio `/app` del contenedor, permitiendo el desarrollo en caliente.
- **app/environment:** Establece variables de entorno. En este caso, `FLASK_ENV` se establece en `development`.

4. Endpoints y servicios.

4.1. Default

Este servicio informa al usuario con un reporte del estado de la aplicación y de la versión del modelo de inferencia alojado en ella. Está definido como una petición de tipo GET que deberá estar compuesta por la URL del servidor y la dirección del endpoint en cuestión. Asumiendo un despliegue en el puerto 0000, el endpoint por defecto se compondría como ***http://172.30.19.97:5000/*** y su funcionalidad principal radica en generar el log inicial de ejecución detallando la versión de la aplicación en uso.

4.2. Inferencia

El servicio de inferencia aloja como tal el modelo de inferencia y se accede bajo una petición POST indicando un payload o body request en la consulta, lo cual se conformaría como la información de entrada para el modelo de tal forma que se pueda llevar a cabo la sesión de inferencia. El endpoint asociado a este servicio se define como ***http://172.30.19.97:5001/models*** y responde a la solicitud con el formato de salida de datos ya expuesto.

El endpoint `http://172.30.19.97:5001/models` permite realizar inferencias sobre el estado probable de un paciente en el futuro con respecto a eventos de estados cardíacos. Este endpoint recibe datos sobre signos vitales y otros parámetros médicos, y devuelve las probabilidades de diferentes estados de salud.

Método HTTP: POST

Ruta: `/models`

4.3. Autenticación

Este endpoint requiere autenticación JWT. Si el usuario no está autenticado, se devolverá un código de estado 401 con el mensaje *Authentication Failed*.

La autenticación requerida tendrá que ser adquirida mediante el endpoint de autenticación **http://172.30.19.97:5000/auth/token**. Este servicio es de tipo **POST**, por lo que requiere que el cliente proporcione los datos de sesión en la consulta. El formato de los datos de sesión se constituye de la siguiente forma:

```
1 {"username": "CPS_DAVID_PEREZ_DEV", "password": "UIS@123FCV"}
```

Listado 4.1. Usuario admin

El servicio de autenticación retorna un token único, el cual tiene una duración de clave válida por 20 minutos. Este token será requerido por los servicios de inferencia y por el servicio de validación de usuario.

4.4. Parámetros de Entrada - Inferencia

El cuerpo de la solicitud de inferencia debe ser un objeto JSON con la siguiente estructura:

```
1 {  
2     "idAtencion": int,  
3     "idSigno": dict,  
4     "nomSigno": dict,  
5     "valor": dict,  
6     "fecRegistro": dict  
7 }
```

Listado 4.2. Parametros de inferencia

- **idAtencion**: Identificador de la atención del paciente.
- **idSigno**: Diccionario con los identificadores de los signos.
- **nomSigno**: Diccionario con los nombres de los signos.
- **valor**: Diccionario con los valores de los signos.
- **fecRegistro**: Diccionario con las fechas de registro de los signos.

4.5. Respuesta

La respuesta será un objeto JSON con la siguiente estructura:

```
1 {  
2   "idAtencion": int,  
3   "inferences": {  
4     "Arresto_Cardiaco": float,  
5     "Bajo_Gasto_Cardiaco": float,  
6     "Sano": float,  
7     "Shock_Cardiogenico": float  
8   },  
9   "State": str  
10 }
```

Listado 4.3. Respuesta de la inferencia

- **idAtencion**: Identificador de la atención del paciente.
- **inferences**: Diccionario con las probabilidades de los diferentes estados de salud:
 - **Arresto_Cardiaco**: Probabilidad de arresto cardíaco.
 - **Bajo_Gasto_Cardiaco**: Probabilidad de bajo gasto cardíaco.
 - **Sano**: Probabilidad de estar sano.
 - **Shock_Cardiogenico**: Probabilidad de shock cardiogénico.
- **State**: Estado único inferido del paciente.

Códigos de Estado

- **200 OK**: Solicitud exitosa. Devuelve las inferencias sobre el estado de salud del paciente.
- **401 Unauthorized**: Autenticación fallida.

4.6. Creación de usuario

El endpoint `http://172.30.19.97:5000/create_user` está diseñado para crear un nuevo usuario en la base de datos. A continuación se presenta una descripción detallada de su funcionalidad:

Funcionalidad del Endpoint

■ Ruta y Método:

- **Ruta:** `http://172.30.19.97:5000/create_user`
- **Método HTTP:** POST
- **Código de Estado:** 201 Created (indica que la solicitud ha tenido éxito y se ha creado un nuevo recurso)

■ Entrada:

- **Dependencia de la Base de Datos (db):** El endpoint recibe una dependencia de base de datos inyectada, la cual se utiliza para interactuar con la base de datos.
- **Solicitud de Creación de Usuario (create_user_request):** Este es un objeto que contiene la información necesaria para crear un nuevo usuario. En particular, incluye:
 - `username`: El nombre de usuario del nuevo usuario.
 - `password`: La contraseña del nuevo usuario.

■ Proceso:

- **Hash de la Contraseña:** Se utiliza `bcrypt_context.hash` para convertir la contraseña proporcionada en un hash seguro antes de almacenarla en la base de datos.
- **Creación del Modelo de Usuario:** Se crea una instancia del modelo de usuario (`Users`) con el `username` y el `hashed_password` generados.

- **Agregar a la Base de Datos:** El modelo de usuario creado se agrega a la sesión de la base de datos mediante `db.add(create_user_model)`.
- **Guardar los Cambios:** Se confirman los cambios en la base de datos mediante `db.commit()`.

■ **Salida:**

- El endpoint no devuelve ningún contenido en el cuerpo de la respuesta, pero el código de estado 201 indica que la creación del usuario fue exitosa.

Solicitud HTTP POST:

POST /create_user

Content-Type: application/json

```
{  
  "username": "nuevo_usuario",  
  "password": "contraseña_segura"  
}
```

Respuesta HTTP:

HTTP/1.1 201 Created

El endpoint `http://172.30.19.97:5000/create_user` se encarga de recibir una solicitud para crear un nuevo usuario, hashear la contraseña de manera segura, crear un modelo de usuario con los datos proporcionados, agregar este modelo a la base de datos y confirmar los cambios, asegurando así que el nuevo usuario se almacene de manera segura y eficiente.

4.7. Generador de sesión

Descripción del Endpoint `http://172.30.19.97:5000/token`

```
1 @router.post("/token", response_model=Token)
2 async def login_for_access_token(form_data:
3     Annotated[OAuth2PasswordRequestForm, Depends()],
4     db: db_dependency):
5     user = authenticate_user(form_data.username, form_data.password, db)
6     if not user:
7         raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
8                             detail='Could not validate user.')
9     token = create_access_token(user.username, user.id,
10                                timedelta(minutes=20))
11     return {'access_token': token, 'token_type': 'bearer'}
```

Funcionalidad

Este endpoint proporciona un servicio de autenticación de usuarios mediante la generación de un token de acceso. A continuación, se describe detalladamente el proceso y funcionalidad de este servicio:

Método POST

Ruta `http://172.30.19.97:5000/token`

Parámetros de Entrada

- `form_data`: Este parámetro es una instancia de `OAuth2PasswordRequestForm`, el cual contiene el nombre de usuario y la contraseña proporcionados por el cliente.
- `db`: Dependencia de la base de datos necesaria para la autenticación del usuario.

Proceso

1. **Autenticación del Usuario**: La función `authenticate_user` se encarga de verificar las credenciales proporcionadas (nombre de usuario y contraseña) contra los datos almacenados en la base de datos.

- Si la autenticación falla, se lanza una excepción `HTTPException` con el código de estado `401 Unauthorized` y un mensaje de error indicando que no se pudo validar el usuario.
2. **Generación del Token de Acceso:** Si la autenticación es exitosa, se genera un token de acceso mediante la función `create_access_token`, el cual incluye el nombre de usuario, el ID del usuario y una validez de 20 minutos.
 3. **Respuesta:** La respuesta del endpoint incluye el token de acceso generado y el tipo de token (`bearer`).

Parámetros de Salida

- `access_token`: Token de acceso generado para el usuario autenticado.
- `token_type`: Tipo de token, en este caso `bearer`.

Excepciones

- `HTTPException 401 Unauthorized`: Se lanza si las credenciales proporcionadas no son válidas.

5. Casos de usuario

Para coordinar esta ejecución, es necesario contar con el motor de Docker instalado en el ambiente de despliegue. Una vez activo el motor de Docker, se debe instanciar la consola en la ruta raíz donde se alojan los archivos Dockerfile, compose.yaml y .dockerignore. Una vez allí, ejecutar las líneas docker-compose up -d. Esto iniciará la aplicación Docker como lo indica el archivo compose.yaml, permitiendo así la configuración de volúmenes y puertos preestablecida.

```
1 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app> docker-compose up
   -d
2 [+] Running 1/2
3 - Network app_default Created

   0.7s
4 Container app-server-1 Started

                                           0.7s
5 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>
```

Una vez en curso, para confirmar la ejecución y/o creación del contenedor se consulta mediante el comando docker ps -a, esto listará los contenedores presentes en la máquina local.

```
1 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app> docker ps -a
2 CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
3              PORTS
4              NAMES
5 18655d182e83   app-server    "/bin/sh -c 'uvicorn"   About a minute ago Up
   About a minute 80/tcp, 0.0.0.0:8000->8000/tcp app-server-1
6 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>
```

Para acceder a los últimos logs de ejecución generados por la aplicación, basta con

ejecutar el comando docker-compose logs y este listara la información obtenida en la consola de ejecución de la aplicación contenerizada.

```
1 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app> docker-compose
  logs
2 server-1 | INFO:      Will watch for changes in these directories: ['/app']
3 server-1 | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to
  quit)
4 server-1 | INFO:      Started reloader process [7] using WatchFiles
5 server-1 | 2024-07-12 16:55:47.016356: I
  external/local_tsl/tsl/cuda/cudart_stub.cc:31] Could not find cuda
  drivers
6 on your machine, GPU will not be used.
7 server-1 | 2024-07-12 16:55:47.108020: E
  external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261] Unable to
  register cuDNN factory: Attempting to register factory for plugin cuDNN
  when one has already been registered
8 server-1 | 2024-07-12 16:55:47.108111: E
  external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607] Unable to
  register cuFFT factory: Attempting to register factory for plugin cuFFT
  when one has already been registered
9 server-1 | 2024-07-12 16:55:47.110305: E
  external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515] Unable
  to register cuBLAS factory: Attempting to register factory for plugin
  cuBLAS when one has already been registered
10 server-1 | 2024-07-12 16:55:47.126095: I
  external/local_tsl/tsl/cuda/cudart_stub.cc:31] Could not find cuda
  drivers
11 on your machine, GPU will not be used.
12 server-1 | 2024-07-12 16:55:47.126534: I
  tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow
```

```

    binary is optimized to use available CPU instructions in
    performance-critical operations.
13 server-1 | To enable the following instructions: AVX2 FMA, in other
    operations, rebuild TensorFlow with the appropriate compiler flags.
14 server-1 | 2024-07-12 16:55:50.532890: W
    tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning:
    Could not find TensorRT
15 server-1 | 2.0.30
16 server-1 | test
17 server-1 | 2024-07-12 16:55:55,043 , INFO , Starting FCV-MLAPI.....
18 server-1 | INFO:    Started server process [9]
19 server-1 | INFO:    Waiting for application startup.
20 server-1 | INFO:    Application startup complete.
21 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>

```

Los warnings obtenidos se refieren a la ausencia de los drivers CUDA en la máquina local host, dado que no es de tipo data center ni es una distribución de tipo GPU. El software CUDA no es compatible ni se ejecutaría de ninguna forma en dicha máquina.

Según los logs de ejecución obtenidos por parte de la aplicación, podría afirmarse que el servidor se construye y se ejecuta con éxito. Para efectuar una prueba, consumir el endpoint de pruebas <http://172.30.19.97:5000/docs> y consultar nuevamente el log de ejecución.

```

1 server-1 | INFO:    172.19.0.1:43496 - "GET /docs HTTP/1.1" 200 OK
2 server-1 | 2024-07-12 17:06:15,561 , INFO , {'url': '/openapi.json',
    'method': 'GET', 'date': '12/07/2024 17:06:15', 'duration':
    0.052373647689819336}
3 server-1 | INFO:    172.19.0.1:43496 - "GET /openapi.json HTTP/1.1" 200 OK
4 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>

```

Obtenida la respuesta HTTP 200, se concluye que el contenedor se ejecuta de manera correcta y relaciona correctamente los puertos de comunicación respectivos de la máqui-

na host y del contenedor.

Para apagar el servidor, solo es necesario ejecutar en la ruta raíz de la aplicación: `docker-compose down`. Esto detendrá la virtualización del ambiente y, por lo tanto, el servidor.

```
1 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app> docker-compose
    down
2 [+] Running 2/2
3     Container app-server-1 Removed                                10.5s
4     Network app_default    Removed
    0.3s
5 PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>
```

5.0.1. Orquestador Docker Swarm La configuración Docker implementada permitirá al usuario servidor desplegar la aplicación mediante el servicio de orquestación de Docker. Swarm es una de las cualidades de los contenedores Docker, ya que permite caracterizar el despliegue de operaciones en términos de CPU límite a utilizar, número de réplicas de la app a implementar, tipo de subred de ser necesario y definición de volúmenes.

Al desplegar usando Swarm, el servidor no contará con solo una instancia de la aplicación, sino con el número de instancias que se le indiquen. Esto permitirá que el despliegue del servicio escale en tiempos de respuesta frente a un número de consultas multitudinales sustancialmente más elevado.

Para configurar el despliegue con Swarm, es necesario contar con los contenedores Docker de la app ya generados. A partir del ID de los contenedores, se parametrizan las acciones del archivo `.yaml`. Para este caso en particular, solo se le indicaría a Swarm que el único contenedor a orquestar es aquel expuesto en el puerto 5001, el cual solo contiene el servicio de inferencia. Para iniciar la implementación con Swarm, es necesario iniciar su red. Para ello, en la carpeta donde se encuentra el archivo `.yaml`, ejecutar:


```
docker swarm init
```

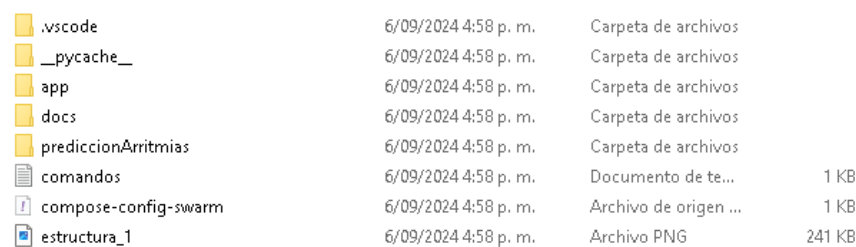
Para desplegar la configuracion .yaml:

```
docker stack deploy -c compose-config-swarm.yml <deploy-name>
```

Una vez desplgada la configuración se podra verificar el estado de los servicios actuales:

```
docker service ls
```

5.0.2. Jerarquia de directorios La aplicación fue seccionada en módulos de trabajo, modelos de datos, bases de datos, requerimientos, logs de ejecución y ejemplos de prueba. En el directorio modules se encuentran los artefactos que constituyen piezas funcionales de la aplicación. Aquí se alojan artefactos como auth.py, predict.py y el modelo de inteligencia artificial desarrollado por CPS.



.vscode	6/09/2024 4:58 p. m.	Carpeta de archivos	
__pycache__	6/09/2024 4:58 p. m.	Carpeta de archivos	
app	6/09/2024 4:58 p. m.	Carpeta de archivos	
docs	6/09/2024 4:58 p. m.	Carpeta de archivos	
prediccionArritmias	6/09/2024 4:58 p. m.	Carpeta de archivos	
comandos	6/09/2024 4:58 p. m.	Documento de te...	1 KB
compose-config-swarm	6/09/2024 4:58 p. m.	Archivo de origen ...	1 KB
estructura_1	6/09/2024 4:58 p. m.	Archivo PNG	241 KB

Figura 3. Distribucion interna

La aplicación se conforma de dos APIs, una de ellas dedicada a la gestión de usuarios y sesiones (app1), mientras que la otra (app2) ejecuta las solicitudes asociadas al servicio de inferencia únicamente. El puerto 5000 fue asignado para la gestión de la seguridad y el puerto 5001 fue destinado para resolver las peticiones asociadas con la inferencia. El directorio app contiene todos los artefactos y subdirectorios que constituyen la seguridad, base de datos, interfaz logger, logs de ejecución, formatos de uso y la aplicación 1 (app1). Los subdirectorios asociados son routes, utils y tests. Dentro de estas rutas se encuentran la configuración de cada una de las funcionalidades de la aplicación a excepción del servicio de inferencia. En la raíz se encuentra el directorio prediccionArritmias destinado para contener los artefactos asociados con la aplicación 2 (app2). El subdirectorio models alojará todas las versiones de inteligencia artificial o modelos .keras que se destinarán tanto para pruebas o producción.

Los archivos de configuración para generar las respectivas imágenes Docker se encuentran en cada uno de los directorios que contienen una aplicación, para este caso en particular, se alojará una configuración Docker para la aplicación contenida en `prediccionArtritmias` y se alojará otra configuración Docker similar en `app` para la aplicación contenida allí.

Los archivos de configuración de despliegue de orquestación (`.yaml`) se encuentran en la raíz.

5.0.3. Integración con el ambiente Dadas las condiciones de infraestructura actualmente adoptadas por el Hospital Internacional de Colombia - HIC, la integración de la aplicación fue hecha bajo una red privada e incluyendo el repositorio completo de la aplicación a su jerarquía de directorios para luego también incluir las imágenes Docker necesarias a su ambiente de pruebas junto con sus respectivos contenedores. Dentro del repositorio de cada aplicación se encuentran dos archivos `.sh` encargados de la creación de imágenes y containerización de dichas aplicaciones.

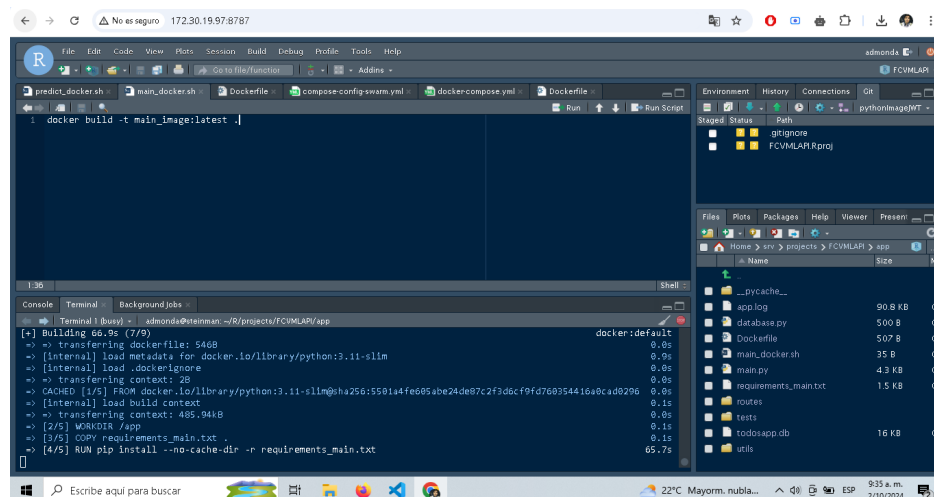


Figura 4. Archivo `main_docker.sh`

La creacion de la imagen encargada del servicio de IA:

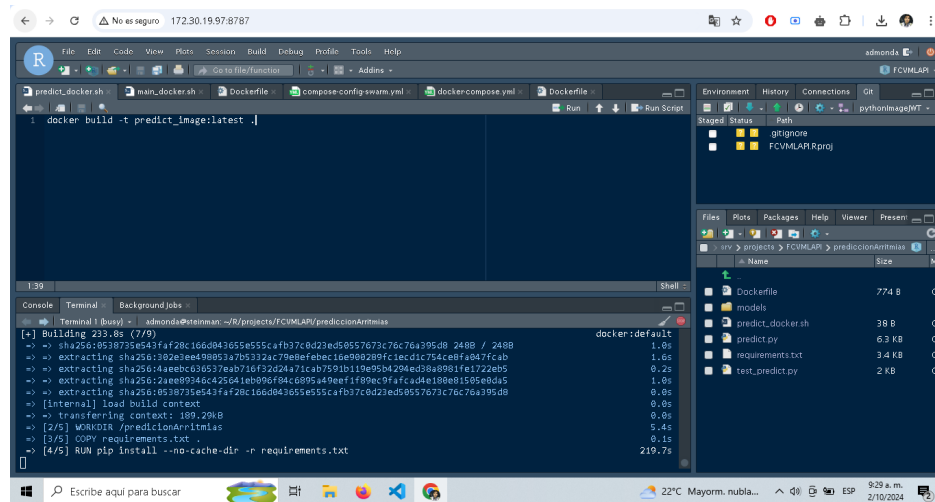


Figura 5. Archivo predict_docker.sh

Configuracion de despliegue main app:

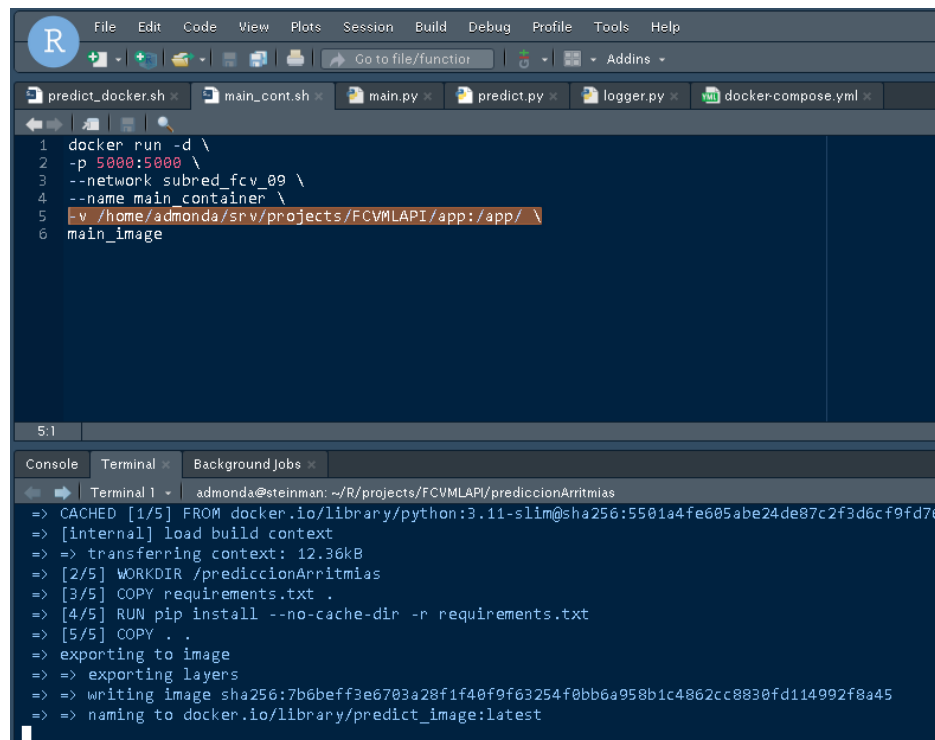
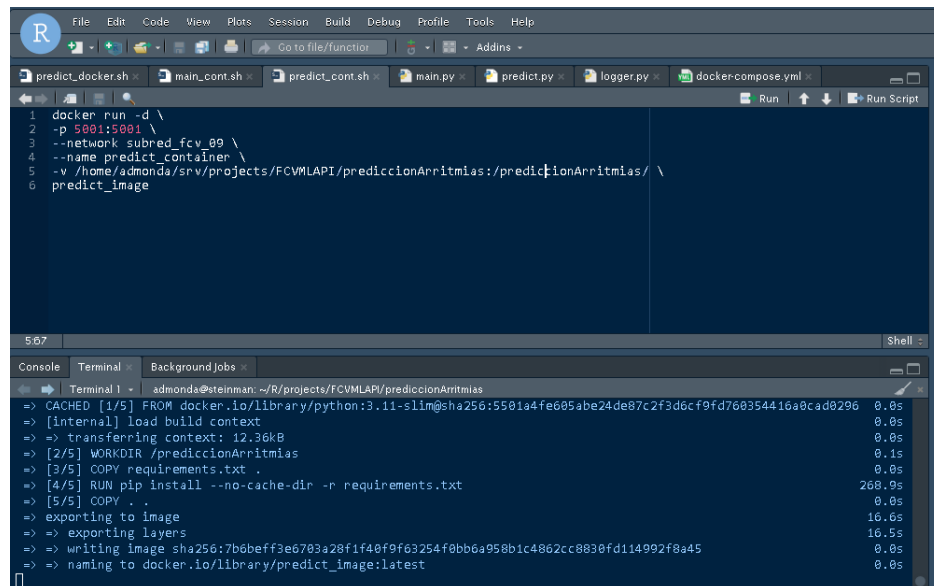


Figura 6. Archivo main_cont.sh

Configuración de despliegue predict app:



```
1 docker run -d \
2 -p 5001:5001 \
3 --network subred_fc_v_09 \
4 --name predict_container \
5 -v /home/admonda/srv/projects/FCVMLAPI/prediccionArtritis:/predictionArtritis/ \
6 predict_image
```

```
=> CACHED [1/5] FROM docker.io/library/python:3.11-slim@sha256:5501a4fe605abe24de87c2f3d6cf9fd760354416a0cad0296 0.0s
=> [internal] load build context 0.0s
=> transferring context: 12.36kB 0.0s
=> [2/5] WORKDIR /predictionArtritis 0.1s
=> [3/5] COPY requirements.txt . 0.0s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt 268.9s
=> [5/5] COPY . . 0.0s
=> exporting to image 16.6s
=> exporting layers 16.5s
=> writing image sha256:7b6beff3e6703a28f1f40f9f63254f0bb6a958b1c4862cc8830fd114992f8a45 0.0s
=> naming to docker.io/library/predict_image:latest 0.0s
```

Figura 7. Archivo predict_cont.sh

La interfaz usada para la implementación de la configuración de forma predeterminada fue R Studio y la ejecución de los contenedores finalmente quedaria implementada dentro de una red de contenedores Docker privada.

5.1. Cliente

Dado el protocolo usado de comunicacion HTTP, se sugiere el consumo de la aplicacion mediante un cliente python, pero no necesariamente tendria que ser un cliente python, podria ser cualquier cliente que pueda resolver peticiones AJAX, esto incluye clientes: js, C++, java, python, html, etc. Para el caso del ejemplo incluido, se construye un cliente python para el consumo de la aplicacion.

5.1.1. El archivo client.py Construido en python y alojado en el directorio test, este cliente primero autentica su sesión para posteriormente realizar una consulta con el archivo input_test.json el cual contiene información de prueba para evaluar el modelo de Deep Learning.

```

1 import requests
2 import json
3
4 # Base URL of the local app
5 base_url = "http://172.30.19.97:5000"
6 predict_url = "http://172.30.19.97:5001"
7
8 # Load the input data for the predict endpoint
9 with open('input_test.json') as f:
10     input_data = json.load(f)
11 print(type(input_data))
12
13 # Authentication credentials
14 auth_data = {
15     "username": "CPS_DAVID_PEREZ_DEV",
16     "password": "UIS@123FCV"
17 }
18
19
20 def get_jwt_token(auth_url, auth_data):
21     try:
22         response = requests.post(auth_url, data=auth_data)
23         print(response.json().get('access_token'))
24         return response.json().get('access_token')
25     except Exception as e:
26         print(e)
27         print("Could not get JWT token")
28         return None
29
30
31
32
33 # Get JWT token
34 token = get_jwt_token(base_url + "/auth/token", auth_data)
35 headers = {'token': token}

```

```

36 print("headers:", headers), print()
37
38 print(), print("Checking the health of the predict service...")
39 response_check = requests.get(predict_url + "/")
40 print(response_check.json())
41 print("="*50)
42
43 print(), print("Checking the health of the main service...")
44 response_check = requests.get(base_url + "/home")
45 print(response_check.json())
46 print("="*50)
47
48
49 response = requests.post(predict_url + "/models",
50                           json=input_data, headers=headers)
51
52
53 print(f"Status Code: {response.status_code}")
54 print(f"Response: {response.json()}")
55 print("="*50)
56
57 response_validate_user = requests.get(base_url + "/", json=headers)
58 print(f"Active User is: {response_validate_user.json()}")
59 print("="*50)

```

Listado 5.1. Archivo Cliente

Este fragmento de código se encarga de interactuar con dos servicios web a través de solicitudes HTTP, utilizando la biblioteca `requests` para realizar operaciones de autenticación y predicción. A continuación, se describe cada parte del código:

1. **Importación de bibliotecas:** Se importan las bibliotecas `requests` y `json`, que son necesarias para realizar solicitudes HTTP y manejar datos en formato JSON, respectivamente.
2. **Definición de URLs:** Se definen las URLs base para la aplicación local y el servicio

de predicción. Estas URLs apuntan a direcciones IP específicas y puertos donde se espera que estén corriendo los servicios.

3. **Carga de datos de entrada:** Se abre un archivo llamado `input_test.json` y se carga su contenido en la variable `input_data`. Este archivo contiene los datos que se utilizarán para realizar una predicción.
4. **Credenciales de autenticación:** Se define un diccionario `auth_data` que contiene el nombre de usuario y la contraseña necesarios para autenticarse en el servicio.
5. **Función para obtener el token JWT:** Se define la función `get_jwt_token`, que realiza una solicitud POST al endpoint de autenticación para obtener un token JWT (JSON Web Token). Este token se utiliza para autenticar las solicitudes posteriores. Si la solicitud es exitosa, se imprime y devuelve el token; si falla, se maneja la excepción y se imprime un mensaje de error.
6. **Obtención del token JWT:** Se llama a la función `get_jwt_token` con la URL de autenticación y las credenciales, y se almacena el token en la variable `token`. Luego, se crea un diccionario `headers` que incluye el token para ser utilizado en las solicitudes posteriores.
7. **Verificación de la salud del servicio de predicción:** Se realiza una solicitud GET al endpoint de predicción para comprobar su estado. Se imprime la respuesta en formato JSON.
8. **Verificación de la salud del servicio principal:** Se realiza una solicitud GET al endpoint principal de la aplicación para comprobar su estado. También se imprime la respuesta en formato JSON.
9. **Solicitud de predicción:** Se envía una solicitud POST al endpoint de modelos del servicio de predicción, enviando los datos de entrada (`input_data`) y los encabezados que incluyen el token JWT. Se imprime el código de estado de la respuesta y el contenido de la respuesta en formato JSON.

10. **Validación del usuario activo:** Finalmente, se realiza una solicitud GET al endpoint principal para validar el usuario activo, enviando los encabezados. Se imprime el resultado de esta validación.

En resumen, este código se encarga de autenticar a un usuario, verificar la disponibilidad de los servicios y realizar una solicitud de predicción utilizando datos de entrada específicos.

Ejecutando el anterior script se debería obtener:

Servidor:

```
server-1 | 2024-07-12 19:27:20,453 , INFO , {'url': '/auth/token', 'method': 'POST', 'date': '12/07/2024 19:27:20',  
server-1 | 'duration': 0.8506975173950195}  
server-1 | INFO: 172.20.0.1:35730 - "POST /auth/token HTTP/1.1" 200 OK  
1/1 [=====] - 1s 642ms/step  
server-1 | Predicción 1: El paciente con id 1408826 se encuentra en estado: Sano  
server-1 | idAtencion  
server-1 | Inference  
server-1 | State  
server-1 | <class 'list'>  
server-1 | 2024-07-12 19:27:21,435 , INFO , {'url': '/predict', 'method': 'POST', 'date': '12/07/2024 19:27:21', 'du  
server-1 | ration': 0.9533584117889404}  
server-1 | INFO: 172.20.0.1:35746 - "POST /predict HTTP/1.1" 200 OK  
PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app>
```

Figura 8. Logs de ejecución - Canal del servidor

Cliente:

```
PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app\test> python client.py  
Status Code: 200  
Response: {'idAtencion': 1408826, 'inferences': {'Arresto_Cardiaco': 0.06022471934556961, 'Bajo_Gasto_Cardiaco': 6.93  
5220397963349e-08, 'Sano': 0.9397750496864319, 'Shock_Cardiogenico': 7.849151728578363e-08}, 'State': 'Sano'}  
PS C:\Users\artur\OneDrive\Documentos\ML_FASTAPI_FCV\app\test>
```

Figura 9. Logs de ejecución - Canal del cliente

5.2. Pruebas de concurrencia y escalabilidad

Para comprobar la eficiencia de la solución frente a diferentes niveles de concurrencia y evaluar su desempeño en términos de CPU, memoria RAM y tiempo máximo de respuesta, se implementaron pruebas de carga en las cuales se buscó evidenciar la respuesta

por parte del servidor en cuanto a especificaciones de latencia y uso de los recursos del hardware.

5.2.1. El archivo `deployment_test.py` Este artefacto fue configurado para ejecutar peticiones de forma asíncrona al servidor de forma parametrizable, permitiéndole al usuario ajustar el número de peticiones que se desean lanzar y la tolerancia (latencia) admitida para esa concurrencia.

```
1 import asyncio
2 import httpx
3 import time
4 from client import get_jwt_token
5 import json
6
7
8
9 base_url = "http://172.30.19.97:5000"
10 predict_url = "http://172.30.19.97:5001"
11
12 auth_data = {"username": "CPS_DAVID_PEREZ_DEV", "password": "UIS0123FCV"}
13 with open("input_test.json") as f:
14     data = json.load(f)
15 token = get_jwt_token(base_url + "/auth/token", auth_data)
16 # headers = {"Authorization": f"Bearer {token}"}
17 headers = {"token": token}
18
19
20 async def send_request(client, data, headers, timeout):
21     try:
22         response = await client.post(predict_url + "/models", json=data,
23                                     headers=headers, timeout=timeout)
24         return response.status_code, response.json()
25     except Exception as e:
26         return None, str(e)
27
28
```

```

29 async def main(num_requests, data, headers, timeout):
30     async with httpx.AsyncClient() as client:
31         tasks = [send_request(client, data, headers, timeout) for _ in
32                     range(num_requests)]
33         start_time = time.time()
34         responses = await asyncio.gather(*tasks)
35         end_time = time.time()
36
37         success_responses = [r for r in responses if r[0] == 200]
38         failed_responses = [r for r in responses if r[0] is None or r[0] != 200]
39         all_responses = [r for r in responses]
40
41         print("All responses:", all_responses)
42         print(f"Total requests: {num_requests}")
43         print(f"Successful responses: {len(success_responses)}")
44         print(f"Failed responses: {len(failed_responses)}")
45         print(f"Total time taken: {end_time - start_time} seconds")
46
47 num_requests = 500
48 timeout = 30 # seconds
49 asyncio.run(main(num_requests, data, headers, timeout))

```

Listado 5.2. Pruebas de carga

Este artefacto de pruebas permite identificar la capacidad actual del servidor en cuanto a la relación petición-respuesta en términos de tiempo, permitiendo comprobar la adaptación a múltiples entradas paralelas.

Estando admitido dentro de la red local, este artefacto puede ser ejecutado desde cualquier máquina y establecer comunicación con la aplicación una vez esta esté autenticada.

5.2.2. Pruebas de integración Esta sección detalla las pruebas realizadas una vez se integró la aplicación a la red y se desplegó el funcionamiento de la misma. Se realizaron diversas pruebas variando tanto la naturaleza del despliegue como el tamaño de carga y tolerancia de espera. Para las primeras pruebas, cada aplicación se desplegó de forma

estándar y no se utilizó el orquestador Swarm.

Se midieron los niveles de consumo de recursos estando el ambiente libre de peticiones y con la aplicación de fondo funcionando en un estado de reposo, se detalla a continuación:

Recursos de ejecucion - estado de reposo:

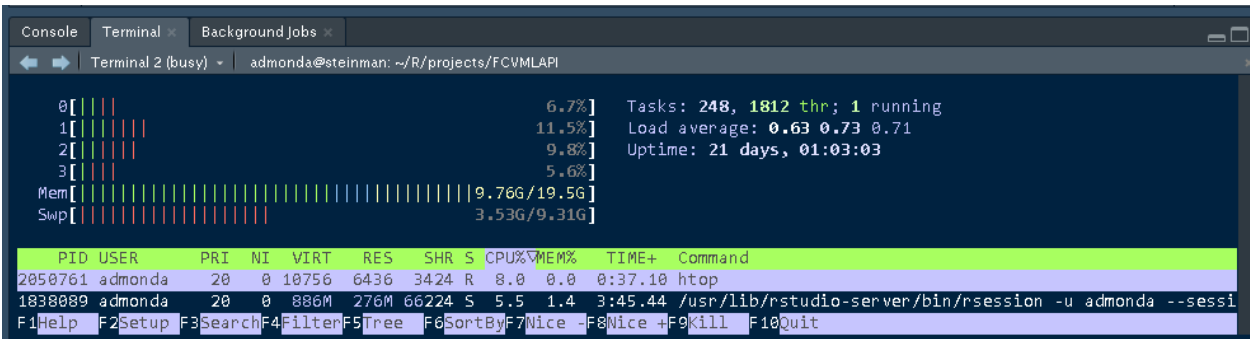


Figura 10. Reporte de recursos de hardware en tiempo real

Para una prueba estandar asincrona de 50 peticiones y un nivel de paciencia de 60 segundos:

Cliente:

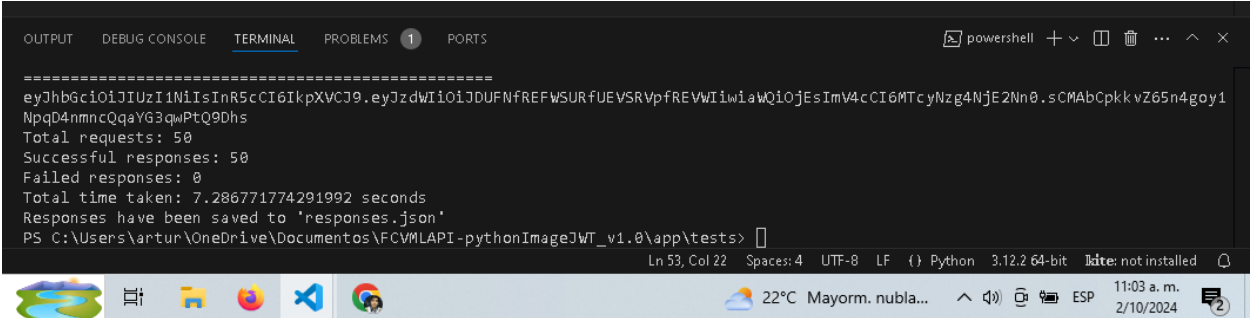


Figura 11. Logs de ejecucion - Canal del cliente

Para esta prueba en particular al ambiente de despliegue le tomo alrededor de 7 segundos procesar 50 consultas.

Al observar el estado de los recursos durante esta ventana de tiempo se observo:

Servidor:

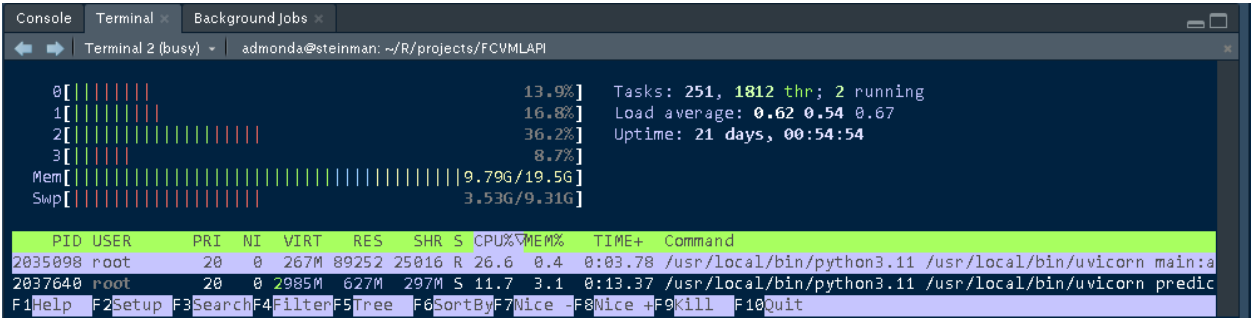


Figura 12. Recursos de ejecucion - Canal del Servidor

Los recursos para un solo core escalaron casi que a un 40 % y de manera proporcional lo hizo la memoria RAM.
 Para una siguiente prueba, se incrementaron las consultas a 200 y se mantuvo el mismo nivel de paciencia:

Cliente:

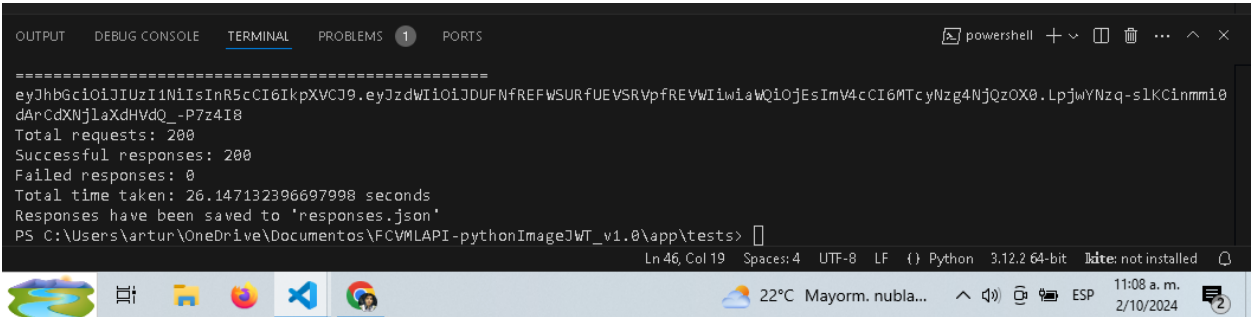


Figura 13. Logs de ejecucion - Canal del cliente

Para esta prueba , al ser mayor el numero de consultas por procesar, el tiempo de res-
 puesta escalo casi a 28 segundos para procesar todas las consultas.
 El impacto en los recursos tambien se pudo observar dentro del rango del tiempo de res-
 puesta.

Servidor:

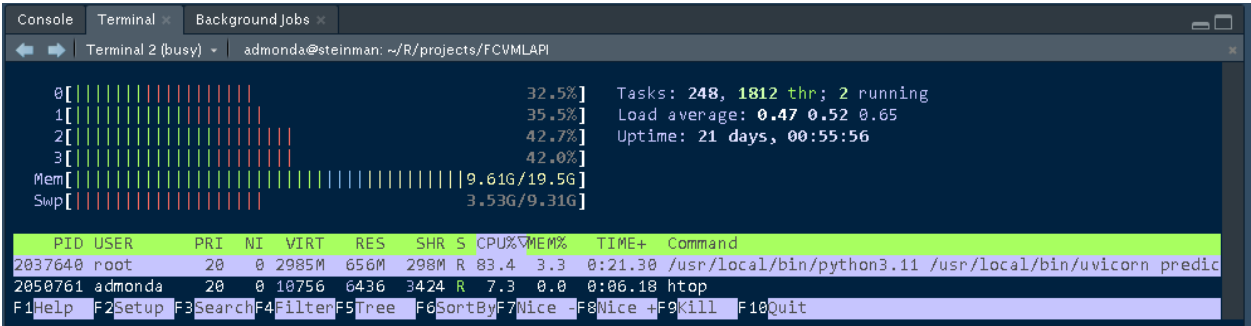


Figura 14. Recursos de ejecucion - Canal del Servidor

Para esta prueba, los recursos de los 4 cores se vieron comprometidos por mas del 35 % todos.

La siguiente prueba contempla un numero de 1000 consultas y una paciencia de 300 segundos que seria el equivalente al cinco minutos.

Cliente:

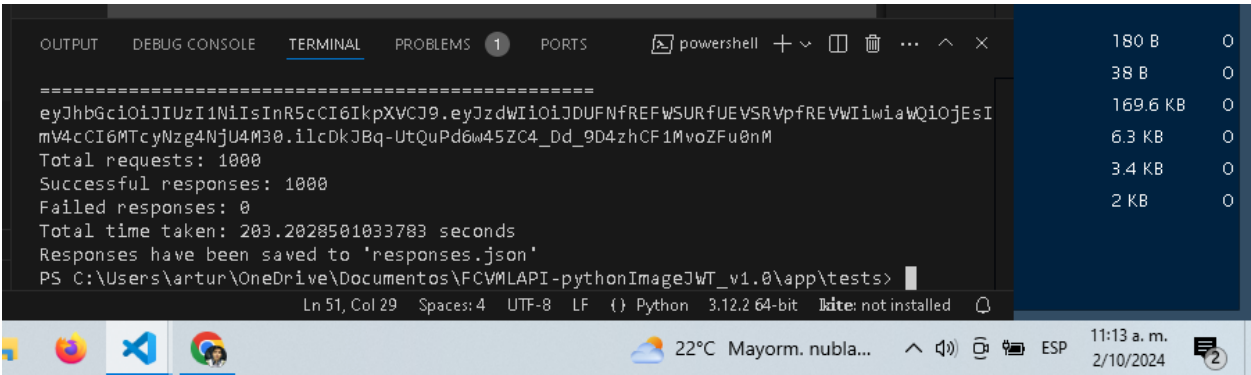


Figura 15. Logs de ejecucion - Canal del Cliente

Se obtuvo una respuesta por parte del servidor al culminar a los 204 segundos. A partir de este nivel de concurrencia los niveles de latencia serian mucho mas notables para el usuario.

Al observar el consumo de recursos, se registro:

Servidor:

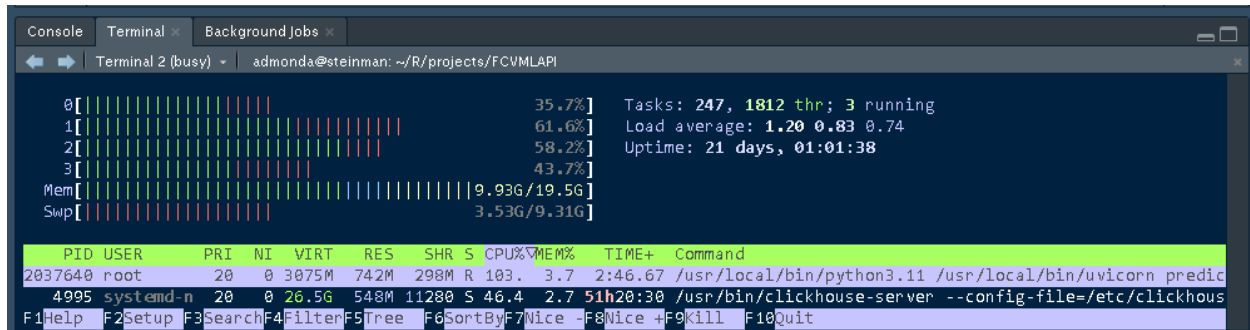


Figura 16. Recursos de ejecución - Canal del Servidor

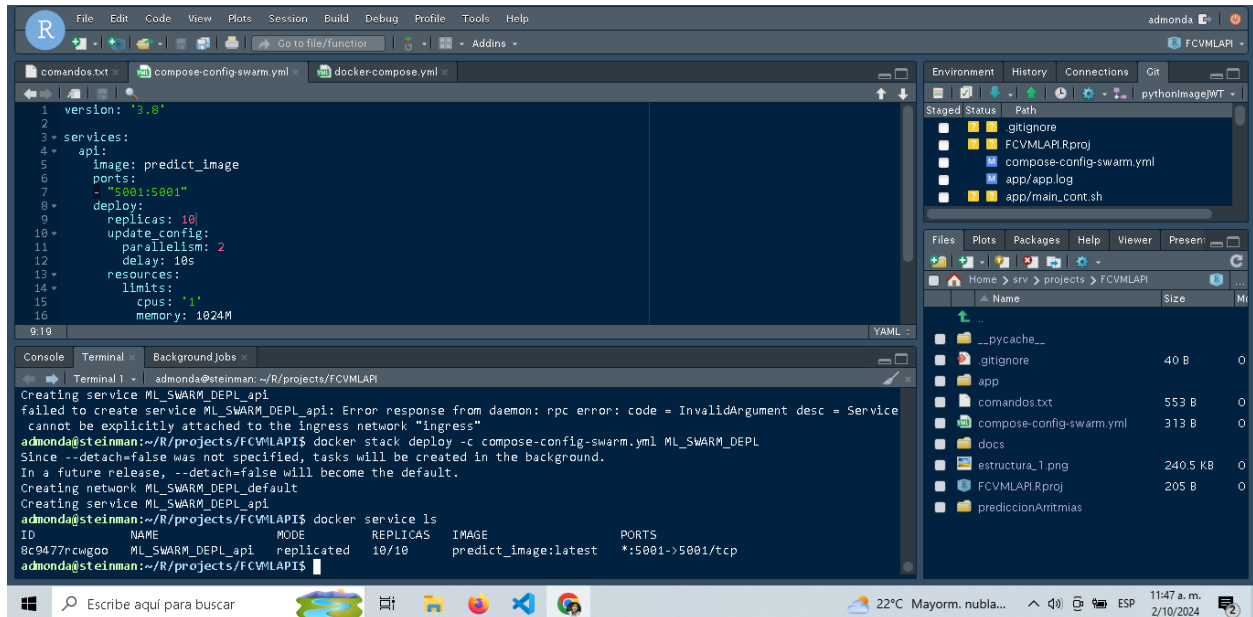
Para este caso podemos observar que la carga promedio es de 52 % y que el mas des-ocupado de sus cores trabaja al 35 % de su capacidad total.

5.2.3. Pruebas de integración con Docker Swarm Ya que se ha puesto a prueba el despliegue convencional de contenedores y se ha analizado y constatado sus características en torno a desempeño y recursos de ejecución, la configuración de despliegue Swarm permitirá observar mejoras en el desempeño y cambios en el consumo de recursos.

Esta configuración de despliegue, como se ha expuesto en la sección de Docker Swarm, explotará las capacidades de hardware del ambiente al implementar contenedores réplica para poder procesar el lote de consultas solicitado en un tiempo sustancialmente más breve, reduciendo así el impacto en la latencia y permitiendo una mayor escalabilidad.

La misma prueba con un lote de 1000 consultas y una paciencia de 300 segundos fue configurada en el archivo de pruebas `deployment_test.py` y se ha liberado el orquestador Swarm con un número de 10 réplicas en el ambiente de despliegue.

Servidor:



```
1 version: '3.8'
2
3 services:
4   api:
5     image: predict_image
6     ports:
7       - "5001:5001"
8     deploy:
9       replicas: 10
10      update_config:
11        parallelism: 2
12      delay: 10s
13      resources:
14        limits:
15          cpus: '1'
16          memory: 1024M
```

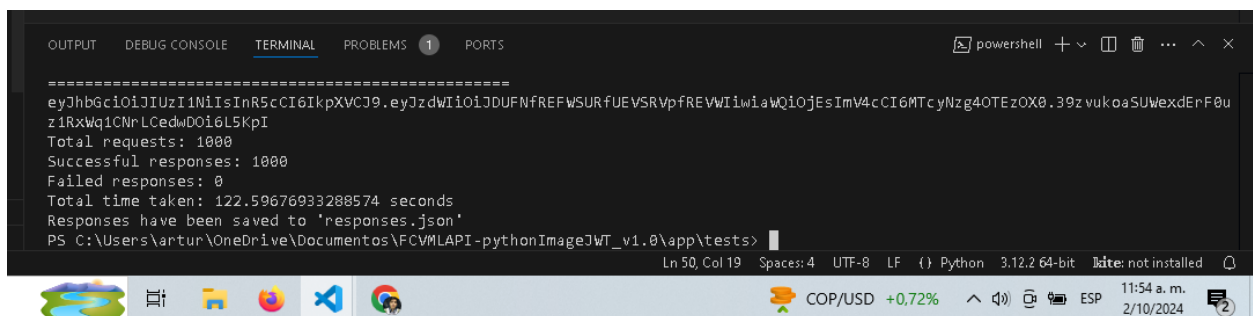
```
admonda@steinman: ~/R/projects/FCVMLAPI
Creating service ML_SWARM_DEPL_api: Error response from daemon: rpc error: code = InvalidArgument desc = Service
failed to create service ML_SWARM_DEPL_api: cannot be explicitly attached to the ingress network "ingress"
admonda@steinman:~/R/projects/FCVMLAPI$ docker stack deploy -c compose-config-swarm.yml ML_SWARM_DEPL
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network ML_SWARM_DEPL_default
Creating service ML_SWARM_DEPL_api
admonda@steinman:~/R/projects/FCVMLAPI$ docker service ls
ID            NAME                MODE                REPLICAS    IMAGE                PORTS
8c9477rcwgo0 ML_SWARM_DEPL_api   replicated          10/10        predict_image:latest *:5001->5001/tcp
admonda@steinman:~/R/projects/FCVMLAPI$
```

Name	Size	MI
__pycache__		0
gitignore	40 B	0
app		0
comandos.txt	553 B	0
compose-config-swarm.yml	313 B	0
docs		0
estructura_1.png	240.5 KB	0
FCVMLAPI Rproj	205 B	0
prediccionArritmias		0

Figura 17. Logs de consola - Canal del Servidor

Al lanzar el lote de 1000 consultas, se obtuvo una respuesta de 122 segundos de duracion aproximadamente.

Cliente:



```
=====
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3ZWIiOiJDUFNfREFWVSRVpFREWVWlwiWQ10jEsImV4cCI6MTcyYnZg40TEzOX0.39zvuKoaSUWexdErF0u
z1RxwQ1CNrLCedwDO16L5KpI
Total requests: 1000
Successful responses: 1000
Failed responses: 0
Total time taken: 122.59676933288574 seconds
Responses have been saved to 'responses.json'
PS C:\Users\artur\OneDrive\Documents\FCVMLAPI-pythonImageJWT_v1.0\app\tests>
```

Figura 18. Logs de ejecucion - Canal del Cliente

El tiempo de ejecución se reduce notablemente al desplegar usando el orquestador swarm y un numero particular de 10 replicas. Para este caso, el consumo de los recursos parece haber sido mucho mas alto que en casos previos:

Servidor:

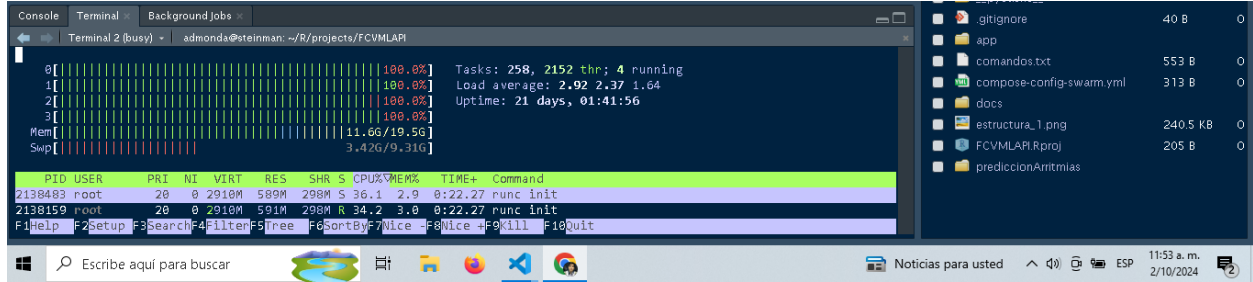


Figura 19. Recursos de ejecucion - Canal del Servidor

En este caso, para resolver las 1000 consultas en 122 segundos, el servidor utilizó su máxima potencia en los 4 cores, paralelizando las cargas y agilizando el proceso, lo que lo convierte peculiarmente flexible frente a futuras implementaciones.